25 Kubernetes Production Issues Every DevOps Engineer Should Know

A practical, hands-on production troubleshooting guide with concrete kubectl commands, YAML examples, and root-cause analysis steps.

Author / Maintainer

Shubham Shirbhate

LinkedIn: https://www.linkedin.com/in/shubham-shirbhate-7b997715a/

YouTube: https://www.youtube.com/@shubhamshirbhate5366

This guide focuses on real-world diagnosis and resolution rather than generic advice. Each issue includes exact commands you can copy-paste into a production cluster.

Table of Contents

1. CrashLoopBackOff

Overview

A pod enters CrashLoopBackOff when its container starts, exits with a non-zero code, and Kubernetes repeatedly restarts it with an exponential back-off delay (10s → 20s → 40s … up to 5 min).

Symptoms

- Pod status shows CrashLoopBackOff or Error.
- Restart count increases continuously (kubectl get pods).
- Application never becomes Ready.

Common Causes

- Application process exits (uncaught exception, missing binary, wrong entrypoint).
- Missing or incorrect environment variables / secrets.
- Failed dependency (database not ready, config file absent).
- Resource limits too low causing immediate OOM or CPU starvation on startup.
- Liveness probe killing the container before it is ready.

Step-by-Step Troubleshooting

1. Identify the failing pod and recent events:
2.

```
kubectl get pods -n <ns> -o wide
kubectl describe pod <pod-name> -n <ns>
kubectl get events -n <ns> --sort-by='.lastTimestamp' | tail -20
```

3. Fetch current and previous container logs (critical for crash loops):
4.

```
kubectl logs <pod-name> -n <ns> --previous
kubectl logs <pod-name> -n <ns> -c <container> --tail=100
```

5. Check exit code and last state:
6.

```
kubectl get pod <pod-name> -n <ns> -o
jsonpath='{.status.containerStatuses[0].lastState.terminated}'
```

7. If the container starts briefly, exec in (only works if not immediately crashing):
8.

```
kubectl exec -it <pod-name> -n <ns> -- /bin/sh
```

9. Validate the Deployment/Pod YAML for correct command, args, env, volumeMounts and probes.
10. Fix the root cause (code bug, missing secret, wrong image tag, probe timing) and redeploy:
11.

```
kubectl apply -f fixed-deployment.yaml
kubectl rollout status deployment/<name> -n <ns>
```

Additional Notes / Root-Cause Checks

- Exit code 137 usually means OOMKilled; 1xx often indicates application error.
- Check initContainers as well — a failing init container also produces CrashLoopBackOff on the main container.

Prevention

- Always set meaningful resource requests/limits.
- Use startupProbe + readinessProbe + livenessProbe with appropriate delays.
- Fail fast in CI with smoke tests against a real cluster or kind/minikube.
- Ship structured logs and enable container log aggregation (Loki, ELK, CloudWatch).

2. ImagePullBackOff

Overview

Kubernetes cannot pull the container image. The pod stays in ImagePullBackOff or ErrImagePull state and never starts.

Symptoms

- Pod status: ImagePullBackOff or ErrImagePull.
- Events show 'Failed to pull image' or 'Back-off pulling image'.

Common Causes

- Incorrect image name, tag, or digest.
- Private registry credentials missing or expired (imagePullSecrets).
- Registry unreachable (network policy, firewall, DNS, rate-limit).
- Image does not exist in the registry or was deleted.
- Node architecture mismatch (e.g. arm64 image on amd64 node).

Step-by-Step Troubleshooting

1. Inspect the exact error:
2.

```
kubectl describe pod <pod-name> -n <ns>
kubectl get events -n <ns> --field-selector involvedObject.name=<pod-name>
```

3. Verify the image reference in the pod spec:
4.

```
kubectl get pod <pod-name> -n <ns> -o jsonpath='{.spec.containers[*].image}'
```

5. Test pull manually from a node or a debug pod that has docker/crictl:
6.

```
# On the node or via debug:
crictl pull <image:tag>
# or
docker pull <image:tag>
```

7. Confirm imagePullSecrets are present and the secret contains valid credentials:
8.

```
kubectl get pod <pod-name> -n <ns> -o jsonpath='{.spec.imagePullSecrets}'
kubectl get secret <secret-name> -n <ns> -o yaml
```

9. For private registries create/update the secret:
10.

```
kubectl create secret docker-registry regcred \
--docker-server=<registry> \
--docker-username=<user> \
--docker-password=<pass> \
--docker-email=<email> -n <ns>
```

11. Reference it in the Deployment under spec.template.spec.imagePullSecrets.

Prevention

- Pin images by digest (sha256:…) in production for immutability.
- Use a pull-through cache or internal registry mirror.
- Automate credential rotation and inject imagePullSecrets via a mutating webhook or service account.
- Scan images and validate tags exist before deploying (CI gate).

3. OOMKilled

Overview

The Linux OOM killer terminates a container because it exceeded its memory limit. Kubernetes reports the reason as OOMKilled and may restart the container (depending on restartPolicy).

Symptoms

- Pod/container status shows OOMKilled.
- Last state terminated with reason OOMKilled and exit code 137.
- Application logs stop abruptly; memory metrics spike to the limit.

Common Causes

- Memory limit set too low for the workload.
- Memory leak in the application.
- Unexpected traffic spike or large in-memory dataset.
- Sidecar or init container competing for the same cgroup limit.

Step-by-Step Troubleshooting

1. Confirm OOMKilled:
2.

```
kubectl describe pod <pod-name> -n <ns> | grep -A5 -i 'OOM\|Last State\|Reason'
```

3. Check current and historical memory usage:
4.

```
kubectl top pod <pod-name> -n <ns> --containers
kubectl get --raw /apis/metrics.k8s.io/v1beta1/namespaces/<ns>/pods/<pod-name>
```

5. Inspect the container's memory limit:
6.

```
kubectl get pod <pod-name> -n <ns> -o jsonpath='{.spec.containers[*].resources}'
```

7. If the limit is too low, raise it (and the request) and redeploy. Example:
8. Example configuration:

```
resources:
requests:
memory: "512Mi"
```

```
limits:
memory: "1Gi"
```

9. Investigate application-level leaks with language-specific tools (pprof, heap dumps, jmap, etc.) while the process is still running.
10. After fix, monitor with:
11.

```
kubectl top pods -n <ns> --sort-by=memory
```

Prevention

- Always set both requests and limits; leave headroom (limit ≈ 1.5–2× typical peak).
- Use Vertical Pod Autoscaler (VPA) in recommendation mode to discover right-sizing.
- Alert on container_memory_working_set_bytes approaching the limit.
- Load-test with realistic data volumes before production.

4. Pending Pods

Overview

A pod stays in Pending state when the scheduler cannot place it on any node. No container is started.

Symptoms

- Pod phase remains Pending for a long time.
- Events show 'FailedScheduling' with a reason (Insufficient cpu/memory, node affinity, taints, etc.).

Common Causes

- Insufficient CPU or memory on all nodes (requests cannot be satisfied).
- Node selectors / affinity / anti-affinity rules that match no nodes.
- Taints on nodes that the pod does not tolerate.
- No PersistentVolume available for a required PVC.
- PodTopologySpreadConstraints too strict.
- Cluster Autoscaler not enabled or at max size.

Step-by-Step Troubleshooting

1. Get the scheduling failure reason:
2.

```
kubectl describe pod <pod-name> -n <ns>
# Look for 'Events' section → FailedScheduling
```

3. Check node capacity and allocatable resources:
4.

```
kubectl get nodes -o
custom-columns=NAME:.metadata.name,CPU:.status.allocatable.cpu,MEM:.status.allocatable.memory
kubectl describe nodes | grep -A10 'Allocated resources'
```

5. Inspect taints and whether the pod has matching tolerations:
6.

```
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.taints}{"\n"}{end}'
kubectl get pod <pod-name> -n <ns> -o jsonpath='{.spec.tolerations}'
```

7. If resources are the problem, either free capacity, lower requests, or add nodes / enable Cluster Autoscaler.
8. Temporary debug: remove restrictive affinity or add a matching toleration, then re-apply.

Prevention

- Size resource requests based on real usage (VPA recommendations).
- Use Cluster Autoscaler or Karpenter for elastic node capacity.
- Keep taints/tolerations and affinity rules documented and reviewed.
- Reserve system capacity with PriorityClasses and ResourceQuotas.

5. Node Not Ready

Overview

A worker node reports status NotReady. The kubelet is either down, unreachable, or the node has critical pressure (disk, memory, PID).

Symptoms

- kubectl get nodes shows NotReady or Unknown.
- Pods on that node become NotReady / Unknown and may be rescheduled after the toleration period.

Common Causes

- kubelet service stopped or crashed.
- Network partition between control-plane and the node.
- DiskPressure, MemoryPressure or PIDPressure conditions.
- Out of disk on /var/lib/kubelet or container runtime storage.
- Certificate expiry on the kubelet.
- Node under heavy load causing kubelet heartbeats to fail.

Step-by-Step Troubleshooting

1. List node conditions:
2.

```
kubectl get nodes
kubectl describe node <node-name>
```

3. Check kubelet status on the node (SSH or cloud console):
4.

```
systemctl status kubelet
journalctl -u kubelet -n 100 --no-pager
```

5. Inspect disk and inode usage:
6.

```
df -h
df -i
du -sh /var/lib/containerd /var/lib/docker /var/log 2>/dev/null
```

7. If DiskPressure, clean unused images and containers:
8.

```
crictl images
crictl rmi --prune
# or docker system prune -af
```

9. Restart kubelet if needed:
10.

```
systemctl restart kubelet
```

11. After recovery verify:
12.

```
kubectl get nodes
kubectl get pods -A -o wide --field-selector spec.nodeName=<node-name>
```

Prevention

- Monitor node conditions (DiskPressure, MemoryPressure) with Prometheus/Grafana alerts.
- Configure image GC thresholds and log rotation.
- Use a proper node monitoring agent (node-exporter, Datadog, etc.).
- Prefer ephemeral-storage requests/limits so the scheduler accounts for disk.

6. Readiness Probe Failed

Overview

When a readiness probe fails, the pod is removed from Service endpoints. Traffic stops flowing to it even though the container may still be running.

Symptoms

- Pod is Running but not Ready (READY 0/1).
- Endpoints object does not list the pod IP.
- Service returns connection refused or no upstream.

Common Causes

- Probe path / port / scheme is wrong.
- Application takes longer to become ready than initialDelaySeconds + failureThreshold × periodSeconds.
- Application is overloaded or returns non-2xx.
- Network policy or firewall blocking the probe from kubelet.
- Probe uses HTTP but app only speaks HTTPS (or vice-versa).

Step-by-Step Troubleshooting

1. Examine probe configuration and recent failures:
2.

```
kubectl describe pod <pod-name> -n <ns> | grep -A20 Readiness
```

3. Check endpoints:
4.

```
kubectl get endpoints <svc-name> -n <ns> -o yaml
```

5. Manually test the probe endpoint from another pod or the node:
6.

```
kubectl exec -it <debug-pod> -n <ns> -- curl -v http://<pod-ip>:<port>/healthz
```

7. Adjust timing or path. Example readinessProbe:
8. Example configuration:

```
readinessProbe:
httpGet:
path: /ready
port: 8080
initialDelaySeconds: 10
periodSeconds: 5
failureThreshold: 3
successThreshold: 1
timeoutSeconds: 2
```

9. Prefer a lightweight /ready endpoint that does not hit the database on every check if possible.

Prevention

- Separate startupProbe (long) from readinessProbe (short).
- Make health endpoints cheap and independent of external dependencies when possible.
- Alert on Ready replicas < desired.
- Test probe behavior under load in staging.

7. Liveness Probe Failed

Overview

A failing liveness probe causes kubelet to kill and restart the container. Misconfigured liveness probes are a frequent source of CrashLoopBackOff.

Symptoms

- Repeated container restarts.
- Events show 'Liveness probe failed' followed by 'Container will be restarted'.
- Application appears healthy from outside but still restarts.

Common Causes

- Probe timeout too aggressive for a busy process.
- Health endpoint itself is blocked or deadlocks under load.
- Probe hits a dependency that is temporarily unavailable.
- Wrong port or path.

Step-by-Step Troubleshooting

1. Inspect the probe and failure messages:
2.

```
kubectl describe pod <pod-name> -n <ns> | grep -A15 Liveness
```

3. Temporarily increase timeout and failureThreshold to confirm the diagnosis:
4. Example configuration:

```
livenessProbe:
httpGet:
path: /healthz
port: 8080
initialDelaySeconds: 30
```

```
periodSeconds: 10
timeoutSeconds: 5
failureThreshold: 6
```

5. Verify the endpoint responds quickly even under load:
6.

```
kubectl exec -it <pod> -n <ns> -- curl -w '%{time_total}\n' http://localhost:8080/healthz
```

7. If the app needs longer to start, use a startupProbe instead of a long initialDelaySeconds on the liveness probe.
8. After correction, watch restart count drop:
9.

```
kubectl get pods -n <ns> -w
```

Prevention

- Never make a liveness probe depend on external services (DB, cache).
- Prefer startupProbe + readinessProbe + conservative livenessProbe.
- Monitor restart counts and set alerts on sudden increases.
- Keep health endpoints trivial (return 200 if the process is alive).

8. DNS Resolution Failure

Overview

Pods cannot resolve Service names or external hostnames. This usually points to CoreDNS (or kube-dns) problems, network policies, or node DNS configuration.

Symptoms

- Applications log 'unknown host' or connection timeouts to service names.
- nslookup / dig from inside a pod fails.

Common Causes

- CoreDNS pods not running or not Ready.
- CoreDNS ConfigMap misconfigured.
- NetworkPolicy blocking traffic to kube-system / CoreDNS.
- Node /etc/resolv.conf pointing to an unreachable upstream.
- DNS search domain or ndots settings causing incorrect queries.

Step-by-Step Troubleshooting

1. Test DNS from a debug pod:
2.

```
kubectl run -it --rm debug --image=busybox:1.36 --restart=Never -- nslookup
kubernetes.default.svc.cluster.local
kubectl run -it --rm debug --image=busybox:1.36 --restart=Never -- nslookup kubernetes.default
```

3. Check CoreDNS status:
4.

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=50
```

5. Inspect CoreDNS ConfigMap:
6.

```
kubectl get configmap coredns -n kube-system -o yaml
```

7. Verify Service and Endpoints for kube-dns:
8.

```
kubectl get svc -n kube-system kube-dns
kubectl get endpoints -n kube-system kube-dns
```

9. If NetworkPolicies exist, ensure they allow UDP/TCP 53 to CoreDNS.
10. Restart CoreDNS if needed:
11.

```
kubectl rollout restart deployment/coredns -n kube-system
```

Prevention

- Monitor CoreDNS latency and error rate.
- Keep CoreDNS replicas ≥ 2 and spread across nodes.
- Avoid overly broad NetworkPolicies that forget DNS.
- Use fully-qualified names (svc.ns.svc.cluster.local) in critical paths to reduce search-path issues.

9. PVC Pending

Overview

A PersistentVolumeClaim stays Pending because no matching PersistentVolume can be provisioned or bound.

Symptoms

- kubectl get pvc shows STATUS Pending.
- Pod that mounts the PVC also stays Pending.

Common Causes

- No default StorageClass and PVC does not specify one.
- StorageClass provisioner is missing or misconfigured (cloud IAM, CSI driver).
- Requested accessMode / storage size cannot be satisfied.
- Quota or resource limit on the namespace.
- Zone / topology constraints prevent binding.

Step-by-Step Troubleshooting

1. Describe the PVC for the exact reason:
2.

```
kubectl describe pvc <pvc-name> -n <ns>
```

3. List StorageClasses:
4.

```
kubectl get storageclass
```

5. Check CSI / provisioner pods:
6.

```
kubectl get pods -n kube-system | grep -E 'csi|provisioner|ebs|pd|azuredisk'
```

7. If using dynamic provisioning, ensure the StorageClass exists and is default (or referenced):
8.

```
kubectl get sc
# Patch default if needed:
kubectl patch storageclass <name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

9. For static PVs, create a matching PV with the correct capacity, accessModes and storageClassName.
10. After fixing, the PVC should transition to Bound; then the pod can start.

Prevention

- Always define a default StorageClass in the cluster.
- Document which StorageClasses are available for each environment.
- Set resource quotas for storage requests.
- Test PVC provisioning in CI against a real (or simulated) CSI driver.

10. PVC Full

Overview

The filesystem inside a PersistentVolume has reached capacity. Writes fail with 'No space left on device'.

Symptoms

- Application logs show ENOSPC or write errors.
- df inside the pod reports 100% usage on the mounted volume.

Common Causes

- Data growth exceeded the requested PVC size.
- Logs, temp files or caches filling the volume.
- No log rotation or data retention policy.

Step-by-Step Troubleshooting

1. Confirm usage from inside the pod:
2.

```
kubectl exec -it <pod-name> -n <ns> -- df -h
kubectl exec -it <pod-name> -n <ns> -- du -sh /* 2>/dev/null | sort -hr | head
```

3. Describe the PVC and underlying PV:
4.

```
kubectl describe pvc <pvc-name> -n <ns>
kubectl get pv
```

5. If the StorageClass supports volume expansion:
6.

```
# Edit PVC and increase spec.resources.requests.storage
kubectl edit pvc <pvc-name> -n <ns>
# Then wait for FileSystemResizePending condition to clear
```

7. Otherwise create a larger PVC, copy data (rsync / velero / application-level), switch the pod to the new claim, and delete the old one.
8. Clean up unnecessary data (old logs, temp files) as a short-term mitigation.

Prevention

- Enable allowVolumeExpansion: true on StorageClasses.
- Alert on volume usage > 80%.
- Implement application-level retention and log rotation.
- Prefer object storage (S3, GCS) for large or unbounded data.

11. High CPU Usage

Overview

Pods or nodes show sustained high CPU, leading to throttling, increased latency, or pod evictions.

Symptoms

- kubectl top shows high CPU percentages.
- Application latency or error rate rises.
- CPU throttling metrics appear (container_cpu_cfs_throttled_seconds).

Common Causes

- Traffic spike or noisy neighbor.
- Inefficient code / infinite loop / busy-wait.
- CPU limit set too low → heavy throttling.
- Missing Horizontal Pod Autoscaler.

Step-by-Step Troubleshooting

1. Identify the hottest pods:
2.

```
kubectl top pods -A --sort-by=cpu | head -20
kubectl top nodes
```

3. Check whether the pod is being throttled:
4.

```
# Via metrics or Prometheus query:
# rate(container_cpu_cfs_throttled_seconds_total[5m])
```

5. Inspect current limits:
6.

```
kubectl get pod <pod> -n <ns> -o jsonpath='{.spec.containers[*].resources}'
```

7. If traffic-related, scale the Deployment or enable/tune HPA:
8.

```
kubectl scale deployment/<name> --replicas=N -n <ns>
kubectl get hpa -n <ns>
```

9. Profile the application (pprof, perf, language-specific tools) to find the hot path.
10. Raise CPU limit if legitimate usage exceeds it, or optimize the code.

Prevention

- Set realistic CPU requests (for scheduling) and limits (for protection).
- Use HPA on CPU or custom metrics.
- Alert on throttling and sustained high CPU.
- Load-test before major releases.

12. High Memory Usage

Overview

Pods consume large amounts of memory, risking OOMKills, node pressure, or performance degradation.

Symptoms

- kubectl top shows high memory.
- Node MemoryPressure condition.
- Increasing RSS / working set over time (possible leak).

Common Causes

- Memory leak.
- Large caches or in-memory datasets.
- Memory limit higher than node capacity leading to over-commit.
- Sidecars adding significant overhead.

Step-by-Step Troubleshooting

1. Find memory-heavy pods:
2.

```
kubectl top pods -A --sort-by=memory | head -20
```

3. Check node pressure:
4.

```
kubectl describe nodes | grep -A5 MemoryPressure
```

5. Inspect working set vs limit:
6.

```
kubectl get pod <pod> -n <ns> -o jsonpath='{.spec.containers[*].resources.limits.memory}'
```

7. For leaks, capture a heap dump or profile while the process is still alive.
8. Right-size limits and consider VPA.
9. If a node is under pressure, cordon and drain after moving critical workloads:
10.

```
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
```

Prevention

- Set memory requests close to actual usage; limits with reasonable headroom.
- Monitor container_memory_working_set_bytes and alert near limit.
- Use VPA or custom recommendations to keep sizing current.
- Prefer streaming / pagination over loading entire datasets into memory.

13. Service Not Reachable

Overview

Clients cannot connect to a Service (ClusterIP, NodePort or LoadBalancer). Traffic never reaches the backing pods.

Symptoms

- curl / connection to the Service IP or DNS name fails.
- Endpoints object is empty or does not contain the expected pod IPs.

Common Causes

- Service selector does not match any pod labels.
- Pods are not Ready (readiness probe failing).
- NetworkPolicy denying the traffic.
- Wrong targetPort or protocol.
- kube-proxy or CNI problem.

Step-by-Step Troubleshooting

1. Verify Service and Endpoints:
2.

```
kubectl get svc <svc> -n <ns> -o yaml
kubectl get endpoints <svc> -n <ns> -o yaml
kubectl get pods -n <ns> --show-labels
```

3. Confirm selector matches:
4.

```
kubectl get pods -n <ns> -l <key>=<value>
```

5. Test connectivity from another pod:
6.

```
kubectl run -it --rm debug --image=busybox:1.36 --restart=Never -- wget -qO-
http://<svc>.<ns>.svc.cluster.local:<port>
```

7. If Endpoints are empty, fix labels or readiness probes.
8. Check NetworkPolicies that might apply:
9.

```
kubectl get networkpolicy -n <ns>
```

10. Inspect kube-proxy logs if the problem is cluster-wide.

Prevention

- Always verify selectors with kubectl get pods --show-labels after deploy.
- Use readiness probes so only healthy pods receive traffic.
- Document required NetworkPolicies for each application.
- Prefer fully-qualified Service DNS names in configuration.

14. Ingress Not Working

Overview

External HTTP/HTTPS traffic does not reach the application through the Ingress resource.

Symptoms

- Browser or curl to the host returns 404, 502, 503 or connection refused.
- Ingress address stays empty or points to a wrong load balancer.

Common Causes

- Ingress controller not installed or not healthy.
- Ingress class annotation / ingressClassName mismatch.
- Backend Service or Endpoints missing.
- TLS secret missing or invalid.
- DNS not pointing to the Ingress controller / load balancer.
- Path or host rule incorrect.

Step-by-Step Troubleshooting

1. Inspect the Ingress object:
2.

```
kubectl describe ingress <name> -n <ns>
kubectl get ingress -n <ns> -o wide
```

3. Confirm the controller is running:
4.

```
kubectl get pods -n ingress-nginx # or istio-system, contour, etc.
kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx --tail=50
```

5. Verify backend Service has Endpoints:
6.

```
kubectl get endpoints <backend-svc> -n <ns>
```

7. Check TLS secret if HTTPS is used:
8.

```
kubectl get secret <tls-secret> -n <ns>
openssl x509 -in <(kubectl get secret <tls-secret> -n <ns> -o jsonpath='{.data.tls\.crt}' | base64
-d) -noout -dates
```

9. Test from outside and from inside the cluster.
10. Common fix – ensure ingressClassName matches the installed controller.

Prevention

- Pin the IngressClass explicitly.
- Monitor Ingress controller metrics (request rate, 5xx, latency).
- Use cert-manager for automatic certificate management.
- Keep a smoke-test that hits the public URL after every deployment.

15. Certificate Expired

Overview

TLS certificates used by Ingress, webhooks, or internal services have expired, causing handshake failures.

Symptoms

- Browser shows certificate warning or clients fail with x509 errors.
- Ingress or API server logs mention certificate expiry.

Common Causes

- cert-manager Certificate not renewing.
- Manually created secret never rotated.
- kubelet / apiserver certificates expired (control-plane).
- Wrong secret referenced by the Ingress or Service.

Step-by-Step Troubleshooting

1. Check certificate expiry:
2.

```
kubectl get secret <tls-secret> -n <ns> -o jsonpath='{.data.tls\.crt}' | base64 -d | openssl x509
-noout -dates -subject
```

3. If using cert-manager:
4.

```
kubectl get certificate -A
kubectl describe certificate <name> -n <ns>
kubectl get certificaterequest,order,challenge -n <ns>
```

5. Force renewal (cert-manager):
6.

```
kubectl delete certificaterequest -n <ns> --all
# or annotate to trigger:
kubectl annotate certificate <name> -n <ns> cert-manager.io/issue-temporary-certificate="true"
--overwrite
```

7. For manually managed secrets, generate a new certificate and update the secret, then restart consumers if necessary.
8. Verify after renewal:
9.

```
curl -vI https://<host> 2>&1 | grep -E 'expire|subject|issuer'
```

Prevention

- Use cert-manager with automatic renewal (renewBefore).
- Alert on certificate expiry < 30 days (Blackbox exporter or cert-manager metrics).
- Never commit private keys to git; use external secret stores or cert-manager.
- Document the certificate inventory for the cluster.

16. Secrets Not Mounted

Overview

An application cannot read the expected Secret (as environment variables or volume). The pod may start but the process fails when it looks for credentials.

Symptoms

- Application logs 'file not found' or 'environment variable not set'.
- kubectl describe pod shows MountVolume.SetUp failed or missing key.

Common Causes

- Secret name or key referenced incorrectly.
- Secret does not exist in the same namespace.
- Optional: false and the key is missing → pod fails to start.
- RBAC prevents the ServiceAccount from reading the Secret (rare with volume mounts).
- Secret type mismatch (e.g. expecting kubernetes.crt but key is different).

Step-by-Step Troubleshooting

1. Confirm the Secret exists and contains the expected keys:
2.

```
kubectl get secret <secret-name> -n <ns> -o yaml
kubectl get secret <secret-name> -n <ns> -o jsonpath='{.data}' | jq 'keys'
```

3. Inspect how the pod consumes it:
4.

```
kubectl get pod <pod> -n <ns> -o yaml | grep -A20 -E 'secret|envFrom|volumeMounts'
```

5. Typical correct volume mount:
6. Example configuration:

```
volumes:
- name: creds
secret:
secretName: my-secret
items:
- key: username
path: user
volumeMounts:
- name: creds
mountPath: /etc/creds
readOnly: true
```

7. Or as environment variable:
8. Example configuration:

```
env:
- name: DB_PASSWORD
valueFrom:
secretKeyRef:
name: my-secret
key: password
```

9. After fixing the reference, restart the pods:
10.

```
kubectl rollout restart deployment/<name> -n <ns>
```

Prevention

- Validate Secret existence and keys in CI before deploy.

- Prefer external secret operators (External Secrets, Secrets Store CSI) for production credentials.
- Avoid putting large binary data in Secrets; use volumes with projected sources carefully.
- Use optional: true only when the application can tolerate missing values.

17. ConfigMap Changes Not Reflected

Overview

Updating a ConfigMap does not automatically restart pods. Applications that read the ConfigMap only at startup continue to use the old values.

Symptoms

- ConfigMap is updated but application behavior is unchanged.
- Mounted files under the volume may update (eventually) but process still has old data in memory.

Common Causes

- Pods were not restarted after the ConfigMap change.
- Application caches configuration at startup.
- ConfigMap is mounted as a volume but the process does not watch for file changes.
- Wrong ConfigMap name referenced.

Step-by-Step Troubleshooting

1. Confirm the ConfigMap content:
2.

```
kubectl get configmap <cm-name> -n <ns> -o yaml
```

3. Force a rolling restart so pods pick up the new data:
4.

```
kubectl rollout restart deployment/<name> -n <ns>
kubectl rollout status deployment/<name> -n <ns>
```

5. Alternative: add a checksum annotation so any ConfigMap change triggers a rollout:
6. Example configuration:

```
spec:
template:
metadata:
annotations:
checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
```

7. If the app supports hot reload, ensure it watches the mounted directory (inotify) or polls.
8. For environment variables derived from ConfigMaps, a restart is always required.

Prevention

- Adopt the checksum/config annotation pattern in all Deployments that consume ConfigMaps.
- Document which settings require a restart versus which are hot-reloadable.
- Prefer a single source of truth (GitOps) so ConfigMap and Deployment change together.
- Consider a configuration controller that automatically triggers rollouts.

18. Deployment Stuck

Overview

A Deployment rollout does not complete. New ReplicaSets never become fully available or old pods are not terminated.

Symptoms

- kubectl rollout status hangs.
- Desired replicas never match available / updated replicas.
- Events show failed progress or probe failures on the new pods.

Common Causes

- New pods fail readiness / liveness probes.
- ImagePullBackOff or CrashLoopBackOff on the new version.
- Insufficient resources → new pods stay Pending.
- maxUnavailable / maxSurge settings too restrictive together with PDB.
- ProgressDeadlineSeconds exceeded.

Step-by-Step Troubleshooting

1. Check rollout status and history:
2.

```
kubectl rollout status deployment/<name> -n <ns>
kubectl rollout history deployment/<name> -n <ns>
kubectl get rs -n <ns> -l app=<label>
```

3. Describe the Deployment and the newest pods:
4.

```
kubectl describe deployment <name> -n <ns>
kubectl get pods -n <ns> -l app=<label> -o wide
```

5. If the new version is bad, roll back immediately:
6.

```
kubectl rollout undo deployment/<name> -n <ns>
kubectl rollout status deployment/<name> -n <ns>
```

7. Common root causes to fix before retrying: image name, probes, resource requests, missing secrets.
8. Inspect the ReplicaSet that is stuck:
9.

```
kubectl describe rs <rs-name> -n <ns>
```

Prevention

- Set a reasonable progressDeadlineSeconds (default 600s).
- Use readiness probes so traffic only moves after the new version is healthy.
- Keep maxSurge ≥ 1 and coordinate with PodDisruptionBudgets.
- Automate canary or blue-green analysis before full promotion.

19. NetworkPolicy Blocking Traffic

Overview

A NetworkPolicy is denying expected traffic between pods or from external sources.

Symptoms

- Connections time out or are refused even though Service and Endpoints look correct.
- Traffic works when NetworkPolicies are removed.

Common Causes

- Default-deny policy with missing allow rules.
- Selector mismatch (podSelector / namespaceSelector).
- Policy applies to the wrong direction (ingress vs egress).
- DNS (port 53) blocked, causing cascading failures.
- CNI plugin does not support NetworkPolicy (or is misconfigured).

Step-by-Step Troubleshooting

1. List all policies that select the pod:
2.

```
kubectl get networkpolicy -n <ns> -o wide
kubectl describe networkpolicy -n <ns>
```

3. Temporary isolation test – scale the policy or add an allow-all for debugging (remove afterwards):
4. Example configuration:

```
# Emergency allow-all (delete after debug)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: allow-all-temp
namespace: <ns>
spec:
podSelector: {}
policyTypes: ["Ingress", "Egress"]
ingress: [{}]
egress: [{}]
```

5. Verify the CNI supports NetworkPolicy (Calico, Cilium, Antrea, etc.).
6. Ensure DNS is explicitly allowed if you have egress rules:
7. Example configuration:

```
egress:
- to:
- namespaceSelector: {}
podSelector:
matchLabels:
k8s-app: kube-dns
ports:
- protocol: UDP
port: 53
- protocol: TCP
port: 53
```

8. After fixing, delete the temporary policy and re-test.

Prevention

- Start with a default-deny and carefully open only required paths.
- Keep NetworkPolicies in the same GitOps repo as the application.
- Test policies in a staging namespace that mirrors production selectors.

- Use a policy visualisation tool (e.g. NetworkPolicy editor, Cilium Hubble).

20. HPA Not Scaling

Overview

HorizontalPodAutoscaler does not increase or decrease the number of replicas despite metric thresholds being crossed.

Symptoms

- kubectl get hpa shows TARGETS or current replicas never change.
- Metrics server or custom metrics adapter missing / unhealthy.

Common Causes

- Metrics Server not installed or not scraping the pods.
- Resource requests not set → CPU utilisation cannot be calculated.
- HPA minReplicas / maxReplicas prevent further scaling.
- Custom metric query returns no data.
- Cooldown / stabilisation window still active.

Step-by-Step Troubleshooting

1. Inspect HPA status:
2.

```
kubectl get hpa -n <ns>
kubectl describe hpa <hpa-name> -n <ns>
```

3. Verify Metrics Server:
4.

```
kubectl get deployment metrics-server -n kube-system
kubectl top pods -n <ns>
```

5. Confirm the target Deployment has resource requests:
6.

```
kubectl get deployment <name> -n <ns> -o
jsonpath='{.spec.template.spec.containers[*].resources.requests}'
```

7. Example working HPA:
8. Example configuration:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: myapp-hpa
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: myapp
minReplicas: 2
maxReplicas: 10
metrics:
- type: Resource
resource:
```

```
name: cpu
target:
type: Utilization
averageUtilization: 70
```

9. Force a scale event for testing by generating load, then watch:
10.

```
kubectl get hpa <name> -n <ns> -w
```

Prevention

- Always set CPU/memory requests on containers that HPA targets.
- Install and monitor the Metrics Server (or Prometheus adapter for custom metrics).
- Set sensible min/max and stabilisation windows.
- Alert when HPA is stuck at maxReplicas under load.

21. etcd Performance Issue

Overview

etcd is the Kubernetes backing store. High latency, leader elections, or disk saturation in etcd cause API server slowdowns and cluster instability.

Symptoms

- API requests become slow or time out.
- etcd logs show 'slow fdatasync', leader changes, or high apply duration.
- kubectl commands hang intermittently.

Common Causes

- Slow or saturated disk (etcd is extremely sensitive to fsync latency).
- etcd running on shared storage or network-attached volumes with high latency.
- Too many objects / large objects causing heavy compaction load.
- Network latency between etcd members.
- Insufficient CPU or memory for etcd.

Step-by-Step Troubleshooting

1. Check etcd health and latency (from a control-plane node or etcd pod):
2.

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
endpoint health -w table
```

3. Check disk latency and space:
4.

```
iostat -x 1 5
df -h /var/lib/etcd
```

5. Compact and defrag if the database has grown large:
6.

```
ETCDCTL_API=3 etcdctl ... compact $(etcdctl ... endpoint status -w json | jq
.[].Status.header.revision)
ETCDCTL_API=3 etcdctl ... defrag
```

7. Monitor etcd metrics: etcd_disk_wal_fsync_duration_seconds, etcd_server_leader_changes_seen_total.
8. If on cloud, move etcd to local NVMe or provisioned IOPS volumes.

Prevention

- Run etcd on dedicated fast local SSDs; never on generic network storage.
- Keep etcd cluster size odd (3 or 5) and monitor leader changes.
- Regular compaction and defragmentation (or enable auto-compaction).
- Alert on fsync latency > 10 ms and on leader elections.

22. API Server Slow

Overview

The Kubernetes API server responds slowly, causing kubectl timeouts, controller lag, and admission webhook delays.

Symptoms

- kubectl commands take many seconds or time out.
- Controllers (deployments, replicasets) reconcile slowly.
- High apiserver request latency metrics.

Common Causes

- etcd performance problems (most common root cause).
- Too many or slow admission webhooks.
- Large numbers of objects (especially events, endpointslices).
- API server CPU/memory starvation.
- Network issues between API server and etcd or clients.

Step-by-Step Troubleshooting

1. Check API server and etcd latency metrics (Prometheus):
2.

```
# apiserver_request_duration_seconds
# etcd_request_duration_seconds
```

3. List validating/mutating webhooks and their latency:
4.

```
kubectl get validatingwebhookconfigurations
kubectl get mutatingwebhookconfigurations
```

5. Temporarily bypass a slow webhook for diagnosis (remove or make failurePolicy=Ignore).
6. Check control-plane resource usage:
7.

```
kubectl top pods -n kube-system | grep -E 'kube-apiserver|etcd'
```

8. Review recent events volume:
9.

```
kubectl get events -A --sort-by='.lastTimestamp' | tail -30
```

10. Scale or tune API server flags only after identifying the bottleneck (usually etcd or webhooks).

Prevention

- Keep admission webhooks fast and highly available; set timeouts ≤ 5s.
- Limit Event object retention and use aggregated logging instead.
- Monitor API server request duration by verb and resource.
- Size control-plane nodes appropriately and keep etcd healthy.

23. Worker Node Disk Full

Overview

A worker node's filesystem (usually the container runtime or kubelet directory) fills up, triggering DiskPressure and preventing new pods from scheduling.

Symptoms

- Node condition DiskPressure=True.
- Pods fail with 'no space left on device'.
- Image pulls fail.

Common Causes

- Container images and layers accumulating.
- Container logs not rotated.
- EmptyDir or hostPath volumes filling the disk.
- Package caches or orphaned volumes.

Step-by-Step Troubleshooting

1. Identify the full filesystem:
2.

```
# On the node
df -h
du -x -h /var/lib 2>/dev/null | sort -hr | head -20
```

3. Prune unused images and containers:
4.

```
crictl images
crictl rmi --prune
crictl ps -a
# or docker system prune -af --volumes
```

5. Clear old logs:
6.

```
journalctl --vacuum-time=2d
find /var/log -name '*.gz' -mtime +7 -delete
```

7. Check kubelet image GC settings (imageMinimumGCAge, high/low thresholds).
8. If EmptyDir is the culprit, identify the pod and increase sizeLimit or move data elsewhere.
9. After cleanup, confirm DiskPressure clears:

10.

```
kubectl describe node <node> | grep -A5 DiskPressure
```

Prevention

- Configure container-runtime and kubelet garbage collection.
- Set ephemeral-storage requests and limits on pods.
- Use a log shipper (Fluent Bit, etc.) and keep local retention short.
- Alert on node disk usage > 80%.

24. Failed Rolling Update

Overview

A rolling update leaves the Deployment in a partially updated or unavailable state. Old and new pods coexist longer than expected or the rollout is marked failed.

Symptoms

- kubectl rollout status returns error or times out.
- Both old and new ReplicaSets have pods; availability drops below threshold.
- ProgressDeadlineExceeded condition.

Common Causes

- New pods never become Ready (probe, image, config errors).
- maxUnavailable too high combined with a PodDisruptionBudget.
- Readiness gates or custom conditions not satisfied.
- Resource shortage preventing new pods from scheduling.

Step-by-Step Troubleshooting

1. Inspect current rollout:
2.

```
kubectl rollout status deployment/<name> -n <ns> --timeout=30s
kubectl get rs -n <ns> -l app=<label>
kubectl describe deployment <name> -n <ns>
```

3. Look at the newest pods' events and logs:
4.

```
kubectl get pods -n <ns> -l app=<label> --sort-by=.metadata.creationTimestamp
kubectl describe pod <newest-pod> -n <ns>
kubectl logs <newest-pod> -n <ns> --previous
```

5. Immediate mitigation – undo the rollout:
6.

```
kubectl rollout undo deployment/<name> -n <ns>
```

7. Fix the underlying issue (image, probes, resources, config) then re-apply and watch:
8.

```
kubectl apply -f deployment.yaml
kubectl rollout status deployment/<name> -n <ns>
```

9. Tune the strategy if needed:

10. Example configuration:

```
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 25%
maxUnavailable: 0
```

Prevention

- Use maxUnavailable: 0 (or a PDB) for zero-downtime requirements.
- Always run readiness probes that accurately reflect traffic readiness.
- Test the new version with a canary or progressive delivery tool before full rollout.
- Set progressDeadlineSeconds and alert on ProgressDeadlineExceeded.

25. Namespace Stuck in Terminating

Overview

A namespace but common operational issue: after kubectl delete namespace, the namespace remains in Terminating state indefinitely because finalizers cannot complete.

Symptoms

- kubectl get ns shows STATUS Terminating for a long time.
- Resources inside the namespace also appear stuck.

Common Causes

- Finalizers on the namespace or on resources inside it never finish.
- API service (e.g. a CRD webhook) is unavailable so the finalizer cannot call it.
- Controller that should remove the finalizer is down or misconfigured.
- Circular dependency between resources.

Step-by-Step Troubleshooting

1. Inspect the namespace and its finalizers:
2.

```
kubectl get namespace <ns> -o yaml
```

3. List remaining resources (including those that may be hidden):
4.

```
kubectl api-resources --verbs=list --namespaced -o name | xargs -n1 kubectl get -n <ns>
--ignore-not-found --show-kind
```

5. Common fix – remove the namespace finalizer (use with caution):
6.

```
kubectl get namespace <ns> -o json | jq '.spec.finalizers = []' | kubectl replace --raw
/api/v1/namespaces/<ns>/finalize -f -
```

7. If a specific resource has a stuck finalizer:
8.

```
kubectl patch <resource> <name> -n <ns> -p '{"metadata":{"finalizers":[]}}' --type=merge
```

9. For stuck CRDs, ensure the corresponding operator/controller is running, or remove the finalizer after confirming data is expendable.

10. After cleanup verify the namespace is gone:

11.

```
kubectl get ns <ns>
```

Prevention

- Avoid adding custom finalizers unless the controller is highly reliable.
- When deleting a namespace that contains CRDs, ensure the operators are healthy first.
- Document the finalizer ownership for every custom resource.
- Prefer graceful deletion with a timeout and a documented force-cleanup procedure.

Quick Reference — Essential Commands

```
kubectl get pods -A -o wide
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous --tail=100
kubectl get events -n <ns> --sort-by='.lastTimestamp'
kubectl top pods -A --sort-by=cpu
kubectl top nodes
kubectl get nodes -o wide
kubectl describe node <node>
kubectl get pvc,pv,sc
kubectl get svc,endpoints,ingress -n <ns>
kubectl get networkpolicy -n <ns>
kubectl rollout status deployment/<name> -n <ns>
kubectl rollout undo deployment/<name> -n <ns>
kubectl get hpa -n <ns>
kubectl api-resources --namespaced -o name
```

About This Guide

This document was revised to replace generic placeholder advice with concrete, copy-paste-ready troubleshooting steps. All original promotional links have been removed. For updates, tutorials and related content follow the links below.

LinkedIn: https://www.linkedin.com/in/shubham-shirbhate-7b997715a/

YouTube: https://www.youtube.com/@shubhamshirbhate5366